ZipperGen Workflow Development and Deployment Manual

Concepts, commands, configuration, runtime, and operations

The ZipperGen Authors

Manual edition: 8 August 2026

Abstract

This manual is what you need after the tutorial. It covers the project's files, the workflow language, inspection and validation, running and testing, models and connectors, deployment and operations, and what to do when something is wrong.

It assumes you can use a terminal and an editor. It does not assume you are a Python or operations engineer, and it does not assume you use a coding agent, though it says where one helps.

Start with the tutorial

`docs/first-workflow.pdf` builds one small application end to end in about seven pages. This manual explains the parts it passes over. If you have not read it, read it first. Most of what follows will then be lookup rather than learning.

The promise

The workflow is always ordinary Python. A coding agent may write or change it, but ZipperGen does the deterministic work: loading it, projecting it for every participant, checking that the result is well formed, running it durably, and deploying it. Opaque model output never becomes a deployed application without passing through visible code you can read.

Contents

1 The project

A ZipperGen project is a directory. `zg init` creates one:

File	In git	Holds
`specification.md`	yes	what the workflow should do, in prose
`workflow.py`	yes	the coordination, as Python
`zippergen.toml`	yes	models, assistants, connectors, entry point, no secrets
`AGENTS.md`	yes	points a coding agent at `zg skill`
`CLAUDE.md`	yes	points Claude Code at `AGENTS.md`
`ZIPPERGEN_HOME`	no	credentials, runs, deployments

The split that matters is the last row. Everything above it travels with the project. The last row describes one machine. A colleague who clones the repository gets the workflow, the specification and the configuration, and supplies their own keys.

1.1 `zippergen.toml`

```
schema_version = 2
name = "email-approval"
specification_file = "specification.md"
workflow_entry = "workflow.py:email_approval"

[providers.connections."openai-main"]
kind = "openai"

[providers.connections."approval-bot"]
kind = "telegram"

[models.configurations."openai-gpt-4o"]
connection = "openai-main"
model = "gpt-4o"

[models.assignments]
default = "mock"

[models.assignments.lifelines]
Writer = "openai-gpt-4o"

[connectors.configurations."approval-chat"]
connection = "approval-bot"
kind = "telegram"
chat_id = "12345678"

[connectors.assignments.lifelines]
User = "approval-chat"
```

It is an ordinary file. Edit it in an editor, or let a coding agent edit it. Nothing in ZipperGen owns it. `workflow_entry` is why most commands do not need you to name the workflow, and

```
zg workflow select workflow.py:email_approval
```

records it without editing the file. With no argument it prints the current one. A project with a single workflow needs no entry at all.

Named configurations exist so several participants can share one and it changes in one place. `configurations` says what a thing is, and `assignments` says who uses it.

1.2 What stays on one machine

API keys	environment variables, or `ZIPPERGEN_HOME`
Telegram bot token	typed once, never an argument
Google credentials	`zg provider authorize` / `accept`
Local model endpoints	site configuration
Runs and deployments	`ZIPPERGEN_HOME`

2 Writing workflows

A workflow is a Python function decorated with `@workflow`. Its body is read, rewritten, and turned into an intermediate representation, so it looks like Python and reads like a protocol, but it is not executed line by line as written.

2.1 Participants and messages

```
User = Lifeline("User")
Writer = Lifeline("Writer")

User(message) >> Writer(message)
```

The left side sends, the right receives. The names in brackets say which variables carry the value at each end. They need not match.

A self-send (`A(x) >> A(y)`) is a local rename, not a message.

2.2 Actions

An action is work one participant does.

`@llm`	a model call with a prompt and declared outputs
`@pure`	ordinary Python, output name and type from the return annotation
`@human`	stops and asks a person: `confirm`, `select`, `input`
`@assistant`	runs a coding agent against a repository
`@planner`	asks a model to generate a sub-workflow, validated before it runs

```
@llm(
    system="You write short, friendly replies.",
    user="Reply to this message:\n\n{message}",
    parse="text",
    outputs=[("draft", str)],
)
def draft_reply(message: str): ...

Writer: draft = draft_reply(message)
```

Several actions by one participant can be grouped:

```
with LLM1:
    agreed = check_agreement(verdict, other_verdict)
    trials = inc_trials(trials)
```

2.3 Decisions

```
if approved @ User:
    User: result = sent(draft)
else:
    User: result = discarded()
```

The `@ User` names the participant who evaluates the condition. It is required, and it is the most important annotation in the language: it decides who broadcasts the outcome and who merely receives it.

Loops work the same way:

```
while (not agreed and trials < MAX_ROUNDS) @ LLM1:
    ...
```

Parenthesise compound conditions

`@` binds more tightly than `not`, `and` and `or`. Write `if (not agreed) @ LLM1:`, not `if not agreed @ LLM1:`.

2.4 Parallel regions and co-regions

Independent branches that may interleave:

```
with parallel:
    with branch:
        Orchestrator(candidate) >> TestRunner(candidate)
        TestRunner: (test_status,) = run_tests(candidate)
        TestRunner(test_status) >> Committer(test_status)
    with branch:
        Orchestrator(candidate) >> Security(candidate)
        Security: (security_status,) = scan_security(candidate)
        Security(security_status) >> Committer(security_status)
```

A co-region is weaker: messages that may arrive in any order.

```
with coregion:
    Analyst_A(a) >> Collector(a_report)
    Analyst_B(b) >> Collector(b_report)
```

2.5 Inputs and outputs

```
def email_approval() -> int:
    ...
    return handled @ User
```

An input belongs to the participant that supplies it. The return says who holds the result. `zg validate` lists all declared workflow inputs. If an interactive run still needs any values, it introduces them under a `Workflow inputs` heading and shows declared defaults.

3 Inspecting and validating

3.1 zg validate

```
zg validate
```

Loads the module, projects the protocol for every participant, checks the deployment declaration, and renders the result back. If it passes, the workflow is well formed and the deadlock-freedom guarantee applies to it.

This is the check to run after every change. It is fast and needs nothing configured.

3.2 zg show

`zg show`	the global protocol
`zg show --communications`	messages only, no action bodies
`zg show --agent NAME`	one participant's local program
`zg show --detail full`	everything, including action definitions
`zg show --format json`	structured, for tooling

`--agent` is the interesting one. It shows what a participant actually runs after projection, which is not what you wrote:

```python
@role('Writer')
def email_approval__Writer() -> None:
    while recv_decision('User'):
        message = recv('User')
        draft = draft_reply(message)
        send('User', draft)
```

The `Writer` is told each round whether to continue, because it has work inside the loop. It has no `if`, because it takes no part in the `User`'s decision. A participant's local program mentions only what concerns it, which is why it cannot wait for a message the decider chose not to send.

Use this when a workflow behaves in a way you did not expect. The global view says what you wrote. The local view says what will run.

4 Running and testing

4.1 One verb

`zg run`	execute once, nothing is kept
`zg run --durable`	record every step, so it can be resumed
`zg run --resume`	continue the most recent unfinished run

A plain run leaves no durable record to resume or inspect. That is deliberate: throwaway runs should not accumulate. When you want to come back to a run, to resume it, inspect it, or answer a human action later, use `--durable`.

Missing inputs are asked for in a terminal. In a script, supply them:

```
zg run --input message="Could we move our meeting to Thursday?" --yes
```

4.2 One configuration pattern

Provider connections, models, coding assistants, and connectors use one small grammar:

```
zg provider configure NAME KIND
zg TYPE configure NAME ...
zg TYPE assign TARGET NAME
zg TYPE check [NAME]
zg TYPE remove NAME
```

An external-service connector uses `connector bind REQUIREMENT NAME` instead of `assign`. The provider connection is the named access path to an external provider: it owns the private credential and any machine-specific endpoint. A model selects a model through one connection; a connector selects a chat, mailbox, sheet, or other resource through one connection. In the examples, `approval-bot` is a Telegram provider connection and `approval-chat` is a connector configuration using it. Bare `zg provider`, `zg model`, `zg assistant`, and `zg connector` show each family. `zg config` shows the complete effective result.

In an interactive terminal, required values may be left out. ZipperGen asks for them and shows known targets and configurations when useful. Thus `zg model configure`, `zg assistant configure`, and `zg connector configure` are guided commands. A script or coding agent must pass the required values explicitly, so it cannot wait for an unseen question. Guided model setup asks for a provider connection and model separately. The Python API is always explicit and never prompts.

4.3 Choosing a model

For a durable project choice, create a named configuration and assign it:

```
zg provider configure openai-main openai
zg provider credential openai-main
zg model configure writer openai-main gpt-4o-mini
zg model assign Writer writer
zg model
```

The credential command prompts without echo. The key is saved in the owner-only `ZIPPERGEN_HOME/workspaces/<project>/development.secrets.json` file on this computer. It is never written to `zippergen.toml`. One provider key may serve several named model configurations. You may instead set the normal environment variable. Several model configurations can share the same provider connection, or use different connections when they need different keys.

`--llm mock`	a placeholder for every action, no key needed
`--llm openai:gpt-4o-mini`	a real provider
`--llm scripted:replies.json`	recorded answers, see below
`--llm-for Writer=openai:gpt-4o`	override one participant or action

Without `--llm`, the assignments in `zippergen.toml` apply. The same resolution is used for plain runs, durable runs and deployments. `--llm` is a global one-command override, so it replaces participant and action assignments. Use `--llm-for` for a narrower temporary override. A deployment stores the resolved model specs in its profile; deploy again after changing the project assignments.

`zg model` displays routing without contacting a provider. `zg model check` verifies readiness and makes a small real provider request without printing the credential. Use `zg model unassign TARGET` before removing a configuration that is still referenced.

4.4 Choosing a coding assistant

An `@assistant` action runs Codex or Claude against a repository. Choose the CLI with the same named configuration and assignment pattern:

```
zg assistant configure coding-agent codex
zg assistant assign Maintainer coding-agent
zg assistant check
```

Assign a participant to cover all of its assistant actions. Use an exact target such as `Maintainer.update_release_notes` when one action needs a different backend. Plain runs, durable runs, and deployments use the same routing. A deployment stores the resolved backends in its profile, so deploy again after changing an assignment.

Backend choice and action permissions are separate. The named configuration chooses Codex or Claude. The `@assistant` declaration still owns its fixed instructions and its `access`, `external_tools`, and `shell` policies. `zg config` shows all of them together. `zg assistant check` verifies that each selected CLI is installed and supports every safety option that ZipperGen relies on. It does not log in. Codex and Claude continue to manage their own authentication.

4.5 Deterministic testing

`mock` answers every action the same way, so a workflow driven by it takes one path and no other. In a consensus protocol that means immediate agreement. Everything behind a decision is unreachable.

Scripted responses fix that:

```
{
  "LLM1.assess":     {"verdict": "yes", "reason": "unmoved"},
  "LLM2.assess":     {"verdict": "no",  "reason": "unmoved"},
  "LLM1.reconsider": {"verdict": "yes", "reason": "unmoved"},
  "LLM2.reconsider": {"verdict": "no",  "reason": "unmoved"}
}
```

```
zg run --llm scripted:never-agree.json
```

Keys are `action` or `Participant.action`. The more specific one wins. Values follow one rule worth learning:

`{...}`	a constant: every call is answered the same way
`[{...}, {...}]`	a finite sequence: exactly these calls are expected

Running past the end of a list is an error, not a silent repeat. That is the point: if a change makes an action run more often than the script expects, the run fails instead of passing quietly.

Use it for branches, not for prompts

Scripted responses exercise *coordination*: this reviewer disagrees, that loop runs three times, this branch is taken. They say nothing about whether a real model would produce good text. Both matter, and they are different questions.

4.6 Durable runs

A durable run records coordination state and completed external-action results in its own SQLite store. An ordinarily interrupted run resumes where it stopped. A crash after an external effect succeeds but before its result is recorded can repeat that effect, so irreversible connectors should use idempotency keys.

```
zg run --durable --llm openai:gpt-4o-mini
# Ctrl-C
zg run status
zg run inspect --agent Writer
zg run --resume
```

Starting another `zg run --durable` discards the older selected development run and creates fresh state. To abandon the selected run, stop its foreground process with Ctrl-C and use `zg run reset`. ZipperGen permanently deletes its record and SQLite state and clears the current-run selection; add `--archive` only when a recoverable private copy is wanted. Reset does not start another run; use `zg run --durable` when one is wanted. If this project's deployment is already running, a foreground run warns before it can compete for Telegram updates, mailbox files, or another external resource.

To watch a live durable run from another terminal, use:

```
zg run inspect --watch --agent Writer
```

The display refreshes in place once per second. Ctrl-C closes the display and does not interrupt the run. Use `--interval SECONDS` when a slower refresh is more appropriate.

`running`	executing now
`waiting`	stopped at a human action
`interrupted`	you pressed Ctrl-C
`failed`	a participant raised an error
`done`	finished, with a result

`interrupted` and `failed` can be resumed. `done` cannot.

5 Models, people, and services

5.1 Models

For development, an environment variable is enough:

```
export OPENAI_API_KEY=sk-...
zg run --llm openai:gpt-4o-mini
```

`openai:MODEL`	`OPENAI_API_KEY`
`anthropic:MODEL` (or `claude:`)	`ANTHROPIC_API_KEY`
`mistral:MODEL`	`MISTRAL_API_KEY`
`ollama:MODEL` (or `local:`)	`OLLAMA_BASE_URL`
`mock`	none

To fix the choice in the project rather than the command line, put it in `zippergen.toml` under `models`. The guided command does this and can also collect the key privately:

```
zg model configure
Model configuration name: writer
Available model providers: openai, anthropic, mistral, local, scripted
Model provider: openai
Model name: gpt-4o-mini
OPENAI_API_KEY (input hidden. Press Enter to configure later):
```

The name and model are project configuration. The key is a site credential. They are deliberately stored in different places.

5.2 Asking a person

A `@human` action stops and waits. Where the question appears depends on configuration:

nothing configured	the terminal running the workflow
a Telegram connector, assigned	a message with buttons

The terminal is right while developing and wrong for a deployment nobody is watching. Two commands, doing two different things:

```
zg provider configure approval-bot telegram
zg provider credential approval-bot
zg connector configure approval-chat approval-bot telegram
zg connector assign User approval-chat
```

The first says *which* chat. The second says *who* is asked there: a participant, or one action with `User.approve_reply`.

The bot token is typed without echo and stays on this machine. The chat id goes in `zippergen.toml` and is committed: a colleague gets the chat, and supplies their own token.

5.3 Reading and writing external services

A workflow declares what it needs:

```
zippergen_connectors = (
    ConnectorRequirement(
        name="call-mailbox", kind="gmail",
        participant="Mailbox", access="read-write",
    ),
)
```

and the project says which one, binding it to that requirement by name:

```
zg provider configure google-work google
zg connector configure inbox google-work gmail \
    --query "is:unread in:inbox"
zg connector bind call-mailbox inbox
zg connector configure records google-work google-sheets \
    --spreadsheet-id 1AbC... --tab Calls
zg connector bind call-records records
```

5.4 Authorizing Google

```
zg provider authorize google-work --scopes gmail.modify,spreadsheets
```

This opens a browser, and prints an encoded result rather than saving it. On the machine that will run the workflow:

```
zg provider accept google-work
```

which asks for the result without echo and stores the credential together with the scopes Google actually granted.

The two halves exist because a server usually has no browser. Authorize on your own computer, then accept on the server. The credential never appears in a command line, so it does not reach shell history.

> **Scopes are checked before deployment**
> Deploying refuses when the granted scopes do not cover what the workflow needs For example, a workflow that marks mail as read needs `gmail.modify`, not `gmail.readonly`. Re-run `authorize` with the missing scopes.

6 Deployment

6.1 Deploy

```
zg deploy
```

The ordinary command writes the bundle, prepares the managed Python environment and service files, checks real dependencies, and starts the service. A failure stops it before starting. Use `--no-start` only when you deliberately want preparation and review to stop before a later `zg deploy start`; it is not a required preliminary step.

After the first time, the same command redeploys the current source:

```
zg deploy                    # redeploy the current source
```

6.2 Operating one

`zg deploy status`	is it running, and where is it
`zg deploy logs`	what it has been doing
`zg deploy inspect`	where each participant's local program is now
`zg deploy inspect --watch`	keep that position open and refresh it in place
`zg deploy trace`	recent protocol events
`zg deploy tasks`	decisions waiting for a person
`zg deploy approve`	answer one
`zg deploy check`	readiness checks, in detail
`zg deploy stop` / `zg deploy restart`	move the service
`zg deploy compact`	prune optional history and rotate logs; refuses while running
`zg deploy reset --yes`	archive durable state and start fresh
`zg deploy remove`	delete it

6.3 What removal keeps

durable store, log	kept	the record of what actually ran
profile	kept	what produced them
secrets	**deleted**	must not be left behind
environment, bundle	deleted	rebuilt by deploying again
service files	deleted	regenerated

`--purge` keeps nothing, including the store. What survives lands in `ZIPPERGEN_HOME/trash/deployments/`, readable only by you. Nothing prunes it, so look occasionally.

6.4 Moving a project to a server

1. Clone the repository. The workflow, specification and `zippergen.toml` come with it.
2. `zg validate` confirms the module loads in that environment.
3. Supply what the machine needs: model keys in the environment, the Telegram token via `connector configure`, Google via `authorize` on your laptop and `accept` there.
4. `zg deploy` prepares, checks and starts the service.

Nothing secret is in the repository, so step 3 is the whole difference between a clone and a running service.

7 Working with a coding agent

ZipperGen ships instructions for coding agents. `zg init` writes an `AGENTS.md` pointing at them and a `CLAUDE.md` that imports those same instructions for Claude Code. `zg skill` prints the full guidance.

An already open coding-agent conversation does not automatically reload that guidance after ZipperGen is upgraded. Start a new session, or explicitly ask the agent to run `zg skill` again. An agent sandbox may also use a different `ZIPPERGEN_HOME`; `zg config` shows the active private state location, and credential readiness should be confirmed in the user's own terminal when the sandbox cannot read it.

```
zg skill                  # what an agent should follow
zg skill --agents-md      # a pointer file for an existing project
```

A reasonable division:

the agent	writes and changes the workflow and specification, validates, runs, inspects projections, prepares deployments
you	credentials, authorization, and starting production

That division is a convention, not a wall: an agent with a shell can run any command you can, and can read any file you can. Credentials are typed rather than passed as arguments, so they stay out of the agent's conversation, your shell history and the repository. But once saved they live in a file under `ZIPPERGEN_HOME` that your user can read.

That distinction is worth keeping straight. Entering a credential interactively protects it *at the moment of entry*. It is not a capability boundary. For a real one, run the agent under a different account, or in an environment that does not carry `ZIPPERGEN_HOME`. Nothing in ZipperGen can supply that, because a process running as you can do what you can do.

The specification is yours and the agent's

ZipperGen does not check that `workflow.py` implements `specification.md`, because it cannot: one is prose. There is no gate, no provenance, and no ceremony around it. Keep it accurate the way you keep a README accurate, and tell your agent to update it before changing behaviour, which the shipped skill does.

8 When something is wrong

8.1 The workflow will not load

`zg validate` reports the exception. Common causes: a missing import, an action used before it is defined, or a compound condition without parentheses (`if (not agreed) @ LLM1:`).

8.2 A participant seems to be waiting forever

Look at what it actually runs:

```
zg show --agent ThatParticipant
```

A projected program cannot wait for a message nobody sends, so if a participant is stuck the cause is usually an action that never returns, such as a model call without a timeout, or a human action nobody has answered. Use `zg run tasks` for a durable run or `zg deploy tasks` for the deployment.

8.3 Only one branch ever runs

You are almost certainly using `--llm mock`, which answers every action identically. Script the responses instead.

8.4 A deployment refuses to start

Starting probes every model, assistant CLI, and connector. The message names what failed. The usual causes are a missing API key on that machine, a Google authorization that does not cover the workflow's scopes, or a Telegram token that was never supplied there.

8.5 A run cannot be resumed or inspected

A plain run keeps nothing. Use `--durable` for a run you want to come back to.

8.6 Connector configured but the question still appears in the terminal

Configuring says which chat. Assigning says who is asked there. Both are needed:

```
zg connector assign User approval-chat
```

9 Command reference

Project	
`init [NAME]`	create a project here
`skill`	print the coding-agent instructions
Inspect	
`validate`	load, project, and check
`show`	render the protocol or one participant
`snapshot` / `diff`	record and compare semantics
Run	
`run`	execute, with `--durable` and `--resume`
`run status/reset/inspect/trace/tasks/approve`	manage, observe, or answer a durable run

Configure	
`config` / `check`	show offline configuration or check all live readiness
`provider configure NAME KIND`	save one named provider identity
`provider credential NAME`	save its key or bot token privately
`provider authorize/accept NAME`	move Google authorization from a browser computer
`model configure NAME CONNECTION MODEL`	save one named model configuration
`model assign TARGET NAME`	route a participant or action
`model unassign TARGET` / `model remove NAME`	remove routing or an unused configuration
`assistant configure NAME BACKEND`	save Codex or Claude under a project name
`assistant assign TARGET NAME`	route an assistant participant or action
`assistant check [NAME]`	verify the selected CLI and required safety options
`assistant unassign TARGET` / `assistant remove NAME`	remove routing or an unused configuration
`connector configure NAME CONNECTION KIND`	save one named destination/resource
`connector assign TARGET NAME`	route human actions
`connector bind REQUIREMENT NAME`	bind an external service
`connector unassign TARGET` / `connector unbind REQUIREMENT`	remove connector routing
`connector check [NAME]` / `connector remove NAME`	check or remove
`completion zsh|bash|fish`	print dynamic shell completion

Deploy	
`deploy [--no-start]`	prepare, check, and normally start
`deploy list/prune`	find deployments and archive orphans
`deploy start/stop/restart`	move the service
`deploy status/logs/check`	operate and diagnose
`deploy inspect/trace/tasks/approve`	observe or answer the deployment
`deploy compact/reset/remove`	prune history and logs, reset state, or delete

Every command takes `--help`. Commands that act on a workflow default to the project's. Commands that act on a deployment default to the most recent.

`zippergen` and `zg` are the same program.

10 Where the guarantee comes from

Projection is syntax-directed. For each decision, every participant is one of three things:

decider	evaluates the condition and broadcasts the outcome
recipient	appears in a branch, receives the outcome and follows it
bystander	appears in neither branch, the decision is erased from its program

The `Writer` in the tutorial is a bystander. Its local program does not mention the approval, so it cannot wait on it or disagree about it.

The main theorem states that the projected local programs produce exactly the behaviours of the global workflow, up to the control messages projection introduces. Deadlock freedom follows for well-formed workflows. Both are machine-checked in Lean 4.

That is the trade the whole system makes: write one protocol, accept the language's restrictions, and get a distributed implementation that cannot deadlock, rather than writing each agent separately and hoping.